

OpenCybele (1.)

Projektová dokumentace

Bedřich Hovorka
xhovor07@stud.fit.vutbr.cz

Úvod

Prvotním záměrem tohoto projektu je seznámit se s prostředím OpenCybele a nejlépe na vhodném příkladě, který využívá většiny jeho zajímavých možností, jej představit. OpenCybele je framework pro paralelní a distribuovanou simulaci založenou na agentech v jazyce Java. Podporuje běh agentů, jejich vzájemnou komunikaci pomocí kanálů. Těch lze využít k implementaci různých způsobů řešení konfliktů mezi agenty. Dalším důležitým prvkem je časovač, jehož pomocí lze sdílet modelový čas mezi agenty a tím je též synchronizovat. Programátor (tvůrce modelu) potřebuje znát jen 5 tříd a jedno rozhraní. Při běhu agentů se využívá reflexe a serializace objektů, aby agenti byli na sobě co nejvíce nezávislí, mohli běžet na různých strojích a vzájemně se identifikovat pomocí řetězců.

Popis řešeného problému – konceptuální model

Je dána síť železničních stanic (viz obrázek) propojených jednokolejnou tratí. Cílem multiagentního systému je snížit pravděpodobnost zahlcení sítě vlaky. Předpokládáme, že systém může určovat pořadí a čas vjezdů do systému, pro pozdržení vlaku. Řeší se především tento problém. Stanice A, B a C jsou vstupy a zároveň výstupy do systému.

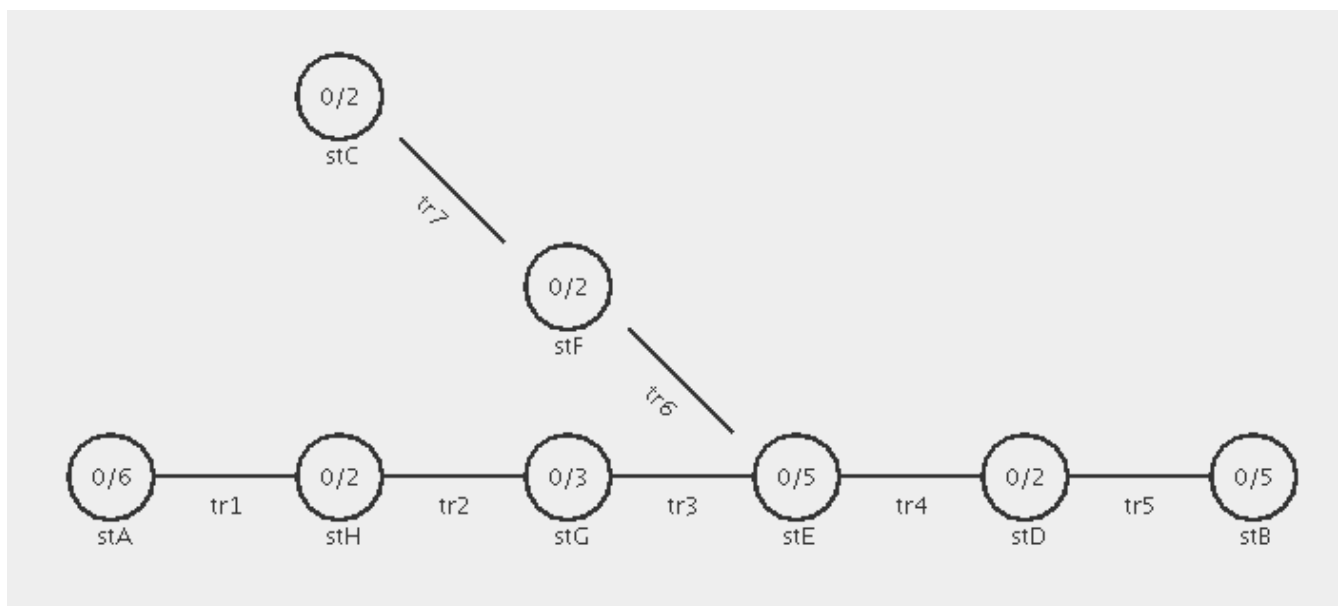


Illustration 1: Náčrt systému

Každá stanice má různou kapacitu a každá trať jinou dobu trvání cesty.

Návrh řešení problému

Řešením je namodelovat SHO pomocí OpenCybele. Hlavní komponentou systému je hlavní agent, který spravuje sociální znalosti, jako je propojení sítě či stavy ostatních agentů. Tyto sociální znalosti slouží též k vizualizaci (některé především). Ta sice zobrazuje jeden konkrétní případ, ale vlastní třídy modelu byly navrženy pokud možno obecně. V systému jsou generovány vlaky žádající o vstup pomocí časovače s exponenciálním rozložením doby příjezdu a náhodným určením startu a cíle z koncových stanic, což je první aktivita hlavního agenta. Druhá aktivita - plánování – je nejsložitější část celého systému – pro každý požadavek:

1. vyhledá se cesta mezi startem a cílem – tj. posloupnost agentů stanic a traťových úseků
2. odhadne se doba příjezdu do každého prvku
3. všem těmto agentům se pošle identifikace vlaku a odhadnutý čas
4. čeká se na odpověď všech agentů na cestě (VoteCollecting), každý pošle rozdíl od odhadu času do času, kdy by si přál, aby vlak přijel (viz dále)
5. ze všech odpovědí se vybere maximum – obálková metoda
6. všem agentům se pošle výsledný odhad času (aby si přidali do svých znalostí)
7. v naplánovaném čase je časovačem vyslán příkaz „start“ vlaku

Tato aktivita inkrementálně vytváří jakýsi „jízdní řád“. Agent stanice hlasuje na základě trošku jiného postupu než agent tratě. Obecně se dá říct, že se „podívají do diáře“ a řeknou nejbližší termín, kdy „mají čas“. Stanice uchovává diář jako multimapu časů na vlaky. V té pak podle své kapacity hledá místo nejprve v intervalu $\langle \text{cas_požadavku} - \lambda, \text{cas_požadavku} + \lambda \rangle$, protože počítá se zpožděním (i jiného vlaku). Jinak je místo za poslední položkou diáře. Obdobně hledá agent trať, ale pro interval používá dobu jízdy. Jinou strukturu znalostí má z důvodu řešení jiného typu konfliktu.

Simulace jízdy vlaku je prováděna pomocí výměny zpráv typu „chci obsadit“, „opouštím“, „začínám jízdu“ a „už si dojel na konec“ mezi vlakem a statickými objekty. Zde nastává další konflikt, kterým je pořadí vstupů do statických objektů. Priorita vstupů do traťového úseku je vypočtena podle rozdílu mezi naplánovaným časem (v diáři) a současným časem.

Instalace, spuštění a návod k použití

Prerekvizity: Java 6, fungující síť (local loopback)

./xhovor07.sh (LINUX),

xhovor07.bat (windows)

Poprvé bude možná třeba spustit 2x, spustí se totiž server a ten nestihne netrpělivého klienta. To platí o všech příkladech. Ovládání viz obrázky.

Závěr

Pravděpodobnost zahlcení je viditelně snížena oproti variantě naivního spuštění simulace bez plánování. Nevím na čí straně je chyba, ale nebylo možné uspokojivě použít synchronní zasílání zpráv tj. metodu Activity.sendAllBlock. Při použití asynchronního zasílání zpráv, zejména v metodě volané časovačem, se stavá, že se zpráva nedoručí a zůstávají „mrtvé vlaky“ v seznamu (a to především při „Fast“ simulaci na windowsech). Také je možné, že to zapříčiní zahlcení systému.

Literatura:
<http://www.opencybele.org/>

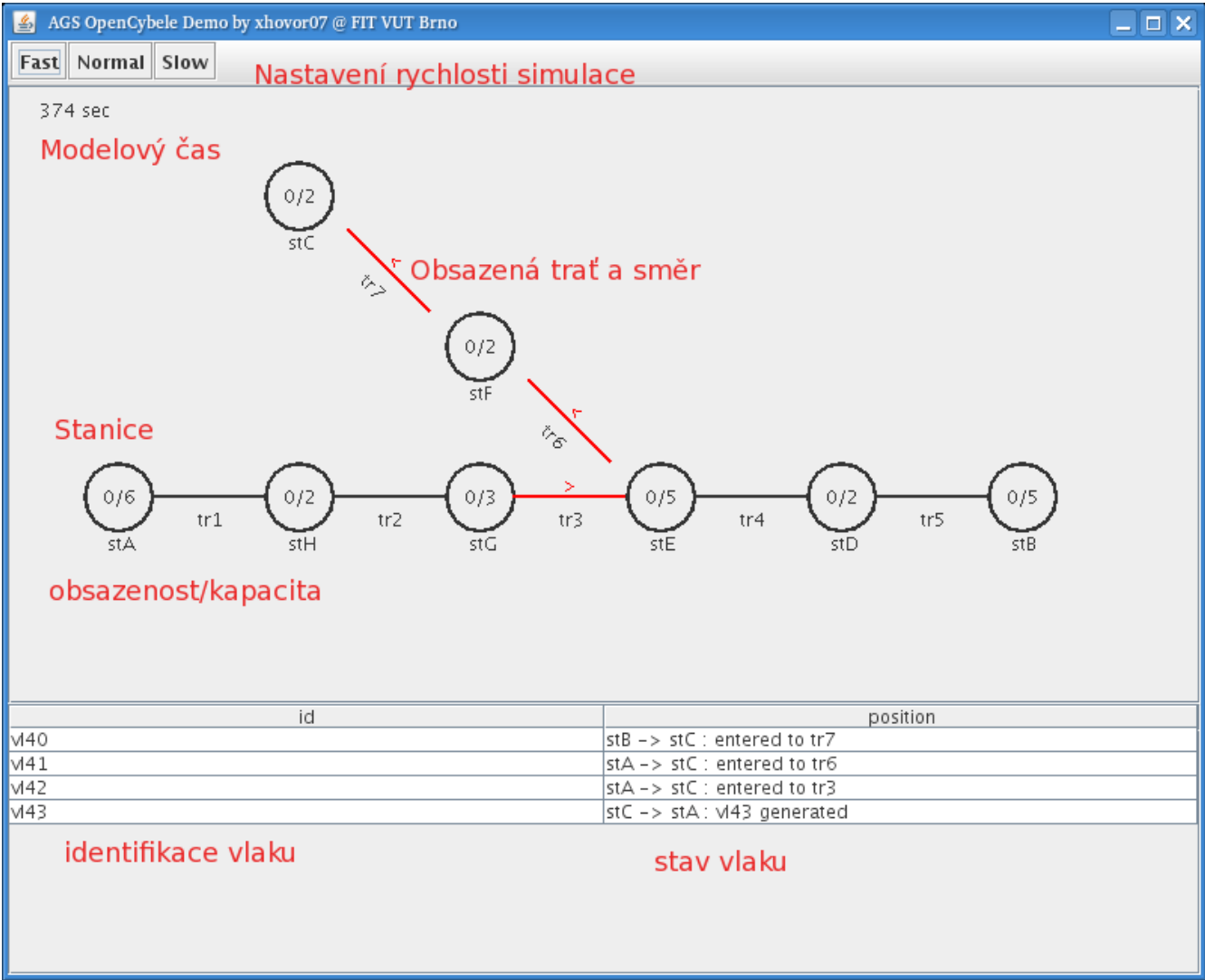


Illustration 2: Rozmístění komponent v GUI

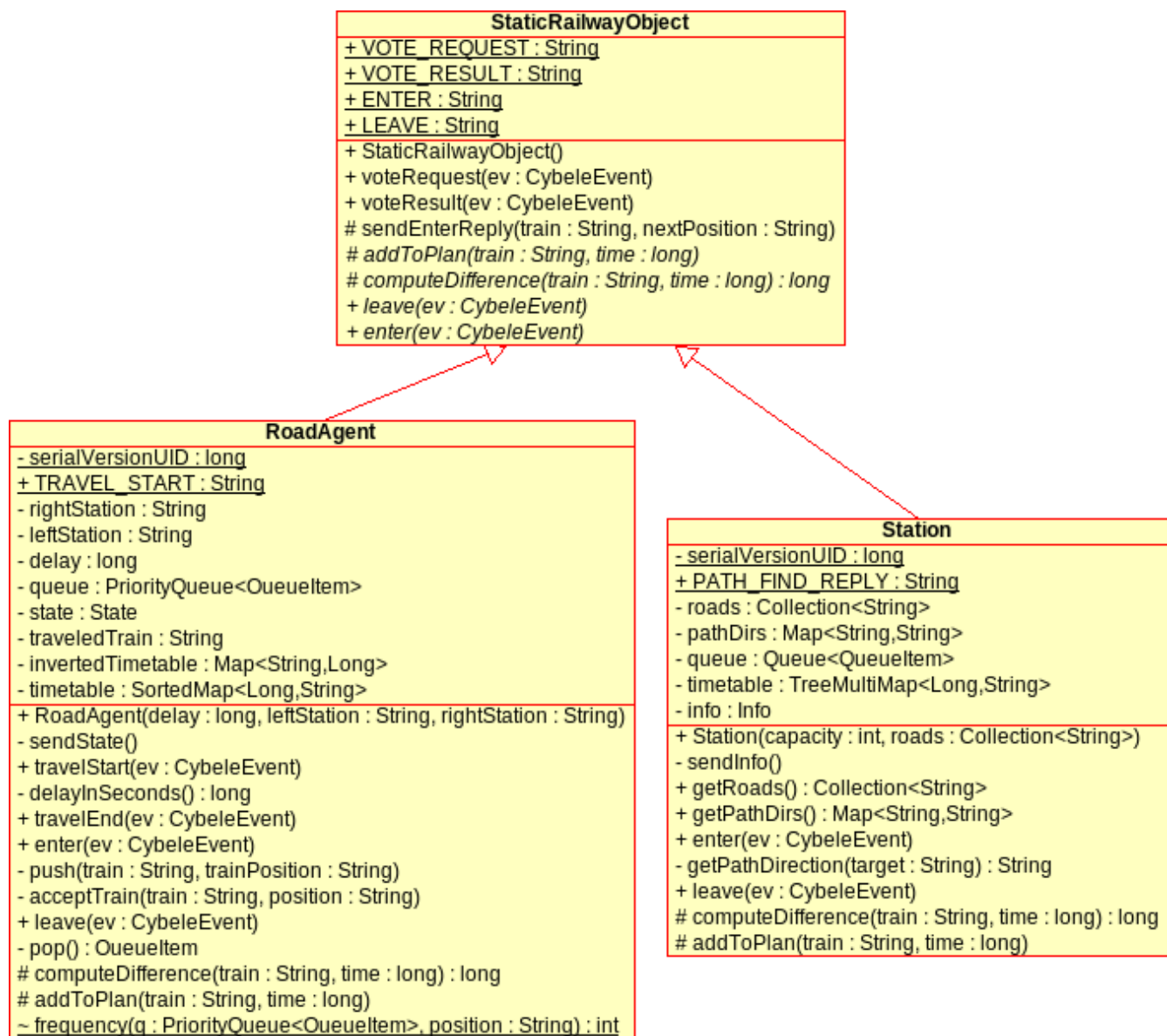


Illustration 3: Diagram tříd statických objektů

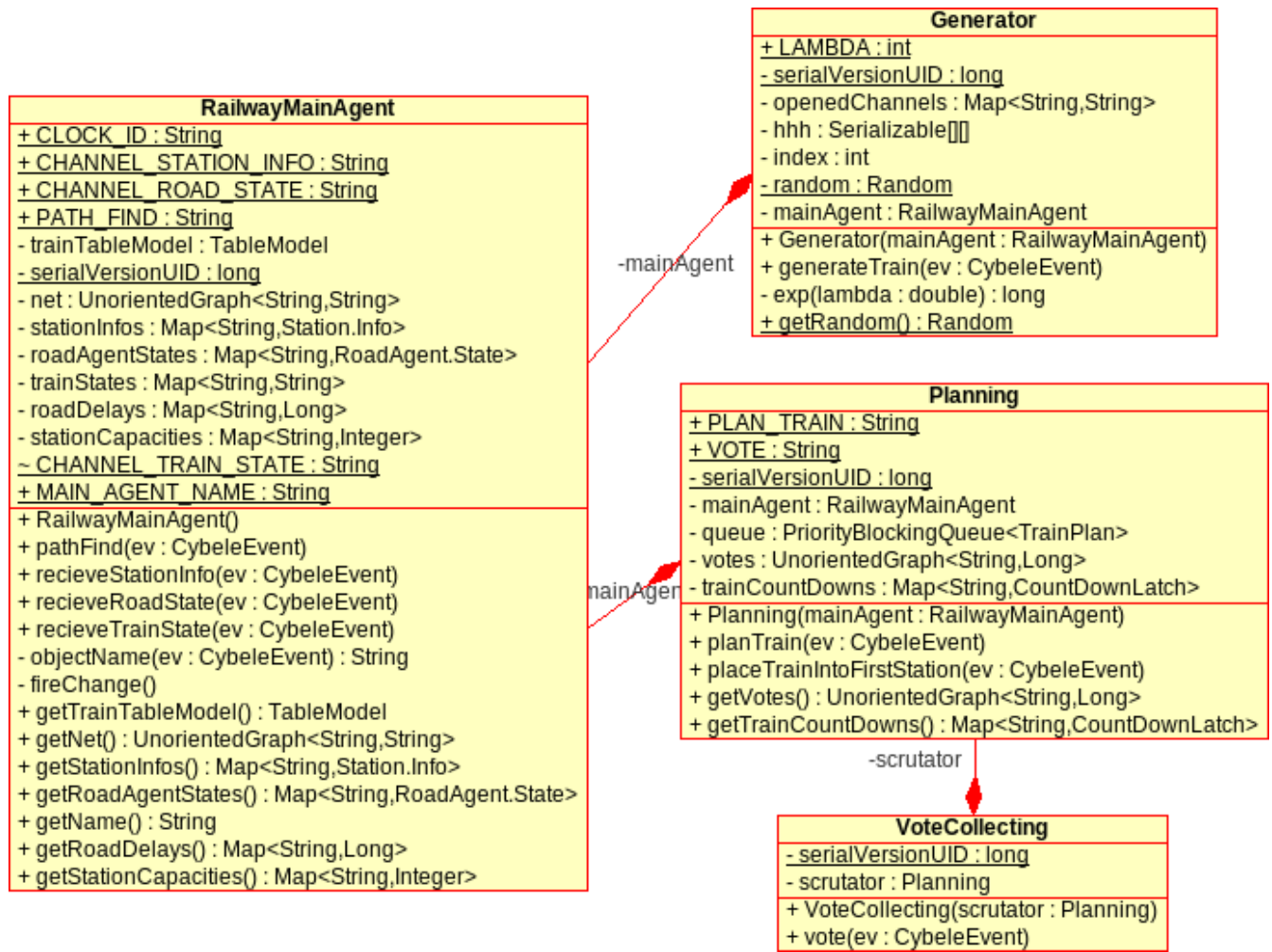


Illustration 4: Hlavní agent a jeho aktivity